Guia técnico — PANTANAL — Uma Saga Antológica

Este documento explica como a aplicação genealógica de antas está organizada, como os arquivos se comunicam, por que a lógica foi construída dessa forma, quais cuidados tomar ao editar manualmente e como publicar a aplicação no GitHub Pages.

A aplicação é um site estático: não há servidor próprio, banco de dados, login ou API obrigatória. O navegador carrega HTML, CSS, JavaScript, SVG e imagens locais. Essa escolha deixa o projeto simples de transportar, compatível com GitHub Pages e fácil de publicar como material complementar do podcast.

1. Visão geral da arquitetura

O fluxo principal é:

```
index.html (raiz do repositório)
        │
        └── encaminha para site/index.html
                                  │
                                  ├── site/css/style.css
                                  ├── site/js/main.js
                                  │        ├── site/js/data.js
                                  │        ├── site/js/graph.js
                                  │        └── site/js/profile.js
                                  └── site/assets/*
```

O navegador abre `site/index.html`. Esse arquivo monta a estrutura visual e carrega `site/js/main.js` como módulo ES. O `main.js` coordena a aplicação: lê os dados, cria o grafo, troca as abas, executa busca, abre fichas, renderiza a novela e conecta os eventos dos botões.

O `data.js` é a fonte estruturada da genealogia. O `graph.js` transforma os dados em um grafo SVG interativo. O `style.css` controla identidade visual, tipografia, desktop, tablet e mobile. As imagens são referenciadas por caminhos relativos dentro de `site/assets`, o que permite que funcionem em uma subpasta do GitHub Pages.

2. O que é cada arquivo

Arquivos da raiz

Arquivo	Função
`index.html`	Página de entrada da publicação. Deve permanecer na raiz para que o GitHub Pages encontre um documento inicial. Ela encaminha o visitante para `site/`.
`package.json`	Declara o nome do projeto, o modo ES Modules, a versão mínima do Node e os comandos `npm run dev`, `npm test` e `npm run build`. Não há dependências externas obrigatórias.
`app.config.ts`	Metadado usado pelo ambiente Manus para a identidade do projeto. Não é necessário para o funcionamento do GitHub Pages.
`scripts/build.mjs`	Remove e recria `dist/`, copiando todo o conteúdo de `site/`. É uma preparação opcional para publicar a pasta compilada; o GitHub Pages pode publicar diretamente a raiz sem esse build.
`.nojekyll`	Informa ao GitHub Pages para não processar o conteúdo com Jekyll. Isso evita que pastas e arquivos estáticos sejam tratados como conteúdo de blog.
`.gitignore`	Impede o versionamento de `node_modules`, `dist`, arquivos `.env`, logs e arquivos locais do sistema.
`README.md`	Documentação resumida para quem clonar ou publicar o projeto. Deve ser mantido alinhado com a aplicação.
`SOURCES.md`	Registra origem dos dados, referência visual e créditos/licenças dos ativos.
`TODO.md`	Histórico de requisitos e entregas do projeto. Serve como memória de escopo, não como código executado.
`GUIA-TECNICO.md`	Este documento. É a explicação detalhada da arquitetura e da manutenção.

Interface e publicação

Arquivo	Função
`site/index.html`	HTML principal: cabeçalho, menus `A rede`, `Personagens` e `A novela`, sidebar de núcleos, área do grafo, modal de vínculos, modal Sobre e carregamento dos scripts.
`site/manus-routes.json`	Manifesto de rota utilizado pelo ambiente de desenvolvimento. Atualmente declara a rota `/`. Não é a lógica de roteamento do GitHub Pages.
`site/.nojekyll`	Cópia do marcador de publicação dentro de `site/`, útil quando o conteúdo dessa pasta é servido diretamente.

JavaScript

Arquivo	Função
`site/js/data.js`	Define indivíduos, gerações, uniões, histórias e estatísticas. Também deriva genitores, filhos, parceiros, irmãos e meio-irmãos.
`site/js/graph.js`	Implementa a classe `GenealogyGraph`. Posiciona os indivíduos, desenha linhas SVG, aplica zoom, permite arrastar o mapa e destaca conexões.
`site/js/main.js`	Faz a integração da aplicação. Renderiza a interface, reage aos cliques, controla abas, busca, núcleos, fichas, modais, exportação e estado selecionado.
`site/js/profile.js`	Valida caminhos de fotos. Aceita caminhos relativos ou URLs HTTPS e rejeita HTTP absoluto e esquemas inseguros.

CSS e imagens

Caminho	Função
`site/css/style.css`	Tema visual Pantanal, Roboto, layout editorial, grafo, cards, modais, desktop, tablet e versão mobile.
`site/assets/pantanal-logo-transparent.webp`	Logo principal transparente usada no cabeçalho e no modal Sobre.
`site/assets/pantanal-logo.jpg`	Versão anterior da identidade visual. Permanece como ativo histórico/alternativo; não é a referência principal atual.
`site/assets/idvisual.webp`	Prancha de identidade visual fornecida para o projeto.
`site/assets/forest.webp`	Fundo visual da sidebar.
`site/assets/tapir-species.jpg`	Foto ilustrativa da espécie, usada como referência visual e não como foto de um indivíduo específico.
`site/assets/favicon.svg`	Ícone da aba do navegador.

Testes

Arquivo	Função
`tests/data.test.mjs`	Verifica IDs únicos, contagens, genitores, irmãos, meio-irmãos, parceiros, histórias e ordens visuais importantes.
`tests/profile.test.mjs`	Verifica as regras de segurança e compatibilidade dos caminhos de fotos.

3. Como os arquivos conversam entre si

3.1 Carregamento inicial

1. O navegador abre `index.html` na raiz.

2. A página de entrada encaminha para `site/index.html`.
3. `site/index.html` carrega `site/css/style.css`, a fonte Roboto e `site/js/main.js` com `type="module"`.
4. `main.js` importa `individuals`, `unions`, `byId`, `getRelations`, `getNetworkIds`, `stories` e `stats` de `data.js`.
5. `main.js` importa `GenealogyGraph` e `initials` de `graph.js`.
6. O grafo é criado com o elemento SVG `#graph` e callbacks de seleção e zoom.
7. A aplicação renderiza o núcleo inicial, a lista de famílias, os personagens e os quatro capítulos da novela.

3.2 Fluxo da rede

Quando a pessoa escolhe um núcleo, `main.js` altera `activeFocus` e chama `renderNetwork()`. Essa função:

1. calcula os IDs visíveis com `currentIds()`;
2. obtém fotos disponíveis para cada indivíduo;
3. chama `graph.render(ids, selectedId, filters, photos)`;
4. atualiza o status da rede;
5. redesenha os botões de núcleos.

O `graph.js` recebe apenas os dados necessários para desenhar. Ele não decide quais vínculos são biologicamente verdadeiros; essa responsabilidade pertence ao `data.js`. O grafo apenas interpreta e apresenta as relações já estruturadas.

Quando a pessoa clica ou toca em um círculo, o SVG chama `onSelect`. Esse callback volta para `highlightNetworkPerson()` em `main.js`. O indivíduo selecionado e suas conexões ficam destacados, sem abrir modal na aba `A rede`.

3.3 Fluxo das fichas

A aba `Personagens` chama `renderCharacters()`. Cada card recebe `data-character-person="id"`. O listener global de cliques em `main.js` identifica esse atributo e chama `openPerson(id, true)`.

A aba `A rede` usa `openPerson(id, false)` apenas para links internos de relação, quando aplicável. Assim, o projeto separa duas experiências:

- na rede: destaque visual e exploração dos vínculos;
- em Personagens: ficha individual com foto disponível e relações conhecidas.

As fotos são resolvidas pela função `photoFor(person)`. O código ainda consegue ler registros locais antigos de `localStorage`, mas a interface atual não oferece mais campos de cadastro ou edição de características. Para publicar uma foto para todo mundo, o caminho correto é editar `data.js` e versionar a imagem em `site/assets`.

3.4 Fluxo da novela

`data.js` exporta o array `stories`, com `id`, `title`, `kicker`, `body` e `focusId`. `main.js` transforma cada item em um card HTML. O botão “Explorar este encontro” usa `focusId` para levar a pessoa à rede no núcleo correspondente.

3.5 Fluxo de publicação

O projeto pode ser publicado sem build, porque o HTML, CSS, JavaScript e imagens já são arquivos estáticos. O script `npm run build` apenas copia `site/` para `dist/`; ele não transpila nem empacota os módulos.

4. Reasoning da lógica usada

4.1 Por que usar dados declarativos

As relações foram mantidas em listas legíveis, em vez de serem codificadas diretamente no SVG. Cada união tem:

```
{
  parents: ['Benjamin', 'Rita'],
  children: ['Doinho', 'Donna'],
  source: 'Benjamin + Rita = Doinho, Donna'
}
```

Isso facilita auditoria: cada linha pode ser comparada com o documento de origem. Também evita duplicar a mesma relação em vários lugares da interface.

4.2 Por que separar nomes de IDs

Os nomes exibidos podem conter acentos, espaços ou alterações editoriais. A função `slugify()` cria IDs estáveis, como:

```
"João Guilherme" → "joao-guilherme"
"Little Bro" → "little-bro"
```

Os IDs são usados em mapas, atributos HTML, URLs de foco e relações. Os nomes continuam sendo a camada de apresentação. Nunca se deve usar o texto exibido como chave manual em um arquivo e o ID em outro sem conferir a conversão.

4.3 Como genitores, filhos e parceiros são derivados

Ao iniciar, `data.js` cria três mapas:

- `parentByChild`: para cada indivíduo, lista os genitores registrados;
- `childrenByParent`: para cada indivíduo, lista os descendentes registrados;
- `partnersByPerson`: para cada indivíduo, lista os parceiros presentes em uma união.

Esses mapas são preenchidos percorrendo `unions`. Dessa forma, a interface não precisa procurar todas as uniões novamente para cada card.

4.4 Como irmãos e meio-irmãos são calculados

Para um indivíduo, o algoritmo percorre seus genitores e encontra outros filhos de cada genitor.

- Se duas pessoas compartilham dois genitores registrados, entram em `siblings`.
- Se compartilham exatamente um genitor registrado, entram em `halfSiblings`, com o ID do genitor compartilhado.

Essa regra é deliberadamente conservadora. Ela não afirma ausência de parentesco biológico; apenas classifica o que está sustentado pelos dados disponíveis.

4.5 Como mãe e pai são identificados

A função `getRelations()` procura entre os genitores alguém com `sex === 'female'` ou `sex === 'male'`. Atualmente, apenas Rita, Justine, Duda e Wie têm sexo feminino confirmado pelo usuário. Como os outros sexos não foram informados, o campo de pai permanece “não informado” quando não há um indivíduo explicitamente marcado como masculino.

Não se infere sexo pelo nome, pela posição no casal ou pela narrativa. Essa decisão evita transformar uma suposição visual em dado científico.

4.6 Como as gerações são usadas

O campo `generation` serve principalmente para organização visual:

- `0`: casais e matriarcas centrais;
- `1`: descendentes e novos núcleos;
- `2`: descendentes posteriores.

Isso não é uma data, idade ou cronologia. A interface usa a geração para criar faixas no grafo, mas não afirma que todos os indivíduos de uma mesma faixa nasceram no mesmo período.

4.7 Por que o grafo é SVG

SVG foi escolhido porque permite:

- desenhar linhas e círculos com precisão;
- adicionar `title`, `role`, `tabindex` e rótulos acessíveis;
- aplicar classes CSS a nós e relações;
- permitir zoom e deslocamento sem biblioteca externa;
- hospedar tudo como arquivo estático.

O grafo calcula posições em uma grade por geração. O espaçamento horizontal é controlado por `step = 146`. As linhas de parceria usam traços; as de genitores usam setas; irmãos e meio-irmãos recebem estilos próprios.

4.8 Por que não abrir modal ao clicar na rede

A rede deve funcionar como mapa de navegação. Abrir uma ficha a cada clique interromperia a leitura das conexões. Por isso, clicar em um círculo apenas aplica destaque e atualiza o status. A ficha completa fica concentrada na aba `Personagens`, onde a intenção de consulta é mais clara.

5. Onde pode dar problema ao atualizar manualmente

5.1 Alterar somente um arquivo de dados

Adicionar um indivíduo em `individualDefinitions` sem adicioná-lo corretamente a uma união pode deixar a anta isolada. O inverso também dá problema: incluir um nome em `unions` sem cadastrá-lo em `individualDefinitions` causa `RangeError: Indivíduo desconhecido`.

Sempre que criar uma pessoa nova:

1. adicione-a a `individualDefinitions`;
2. defina uma geração coerente;
3. use o nome exatamente nas listas `parents` e `children`;
4. rode `npm test`;
5. confira a rede e a aba Personagens.

5.2 Reutilizar nomes

Os IDs são derivados dos nomes. Dois animais diferentes chamados, por exemplo, “Bandaid” gerariam a mesma chave e seriam mesclados. Se houver homônimos, o modelo precisa ser ampliado para usar um identificador explícito, como `Bandaid-1` e `Bandaid-2`, sem confiar apenas em `slugify()`.

5.3 Alterar o nome de um indivíduo

Mudar `Little Bro` para outro texto também muda seu ID. Isso pode quebrar:

- `unions` que usam o nome antigo;
- a ordem visual em `graph.js`;
- testes que procuram `little-bro`;
- links e URLs com `nucleo=`;
- dados salvos anteriormente no navegador.

Se a mudança for apenas de grafia, revise todos esses locais e rode uma busca global pelo nome antigo:

```
grep -RIn "nome-antigo\|id-antigo" site tests README.md TODO.md
```

5.4 Alterar a ordem do grafo

A ordem dos fundadores e da segunda geração está em arrays no `graph.js`. Alterar a lista muda a posição visual, mas não muda a genealogia. Se alguém trocar a ordem sem atualizar os testes, `npm test` pode falhar — isso é desejável, pois o teste protege decisões visuais que foram solicitadas explicitamente.

5.5 Criar união com um filho inexistente

O algoritmo exige exatamente dois genitores por união. Uma união com um genitor repetido, um genitor inexistente ou uma criança não cadastrada quebra a consistência. A parceria Donna + Namorado é um caso válido sem filhos: `children: []` significa que não há descendentes informados, não que a parceria não existiu.

5.6 Inventar sexo, pai, mãe ou cronologia

A interface mostra “não informado” quando o documento não sustenta uma conclusão. Não marque alguém como homem ou mulher apenas pelo nome ou pelo papel narrativo. Se houver nova confirmação, altere o campo `sex`, atualize `matriarchIds` se necessário e acrescente um teste.

5.7 Alterar caminhos de imagens

Os caminhos de imagem são relativos à página HTML, não ao arquivo JavaScript. Para uma foto em `site/assets/individuos/rita.jpg`, o campo deve usar:

```
photo: './assets/individuos/rita.jpg'
```

Não use `site/assets/...` dentro do HTML publicado em `site/`, pois isso duplicaria o prefixo. Também não use links HTTP em uma página HTTPS: navegadores podem bloquear conteúdo misto. O `profile.js` rejeita esse caso.

Fotos de indivíduos precisam ter autorização de uso e devem ser nomeadas de forma rastreável. A foto ilustrativa da espécie não deve ser atribuída a Rita, Benjamin ou qualquer outro indivíduo.

5.8 Alterar o CSS sem considerar os breakpoints

O CSS possui regras para desktop, tablet e mobile. Uma regra escrita no final do arquivo pode sobrescrever uma regra anterior. Isso é especialmente importante no cabeçalho, no grafo e no botão Podcast.

Depois de editar `style.css`, confira pelo menos 390 px e 1440 px. Procure por:

- cabeçalho cortado;
- menus que desaparecem;
- sidebar que invade o conteúdo;
- grafo sem rolagem horizontal;

- modal maior que a tela;
- texto ilegível no SVG.

5.9 Editar `main.js` sem atualizar o HTML

Os seletores usados em `main.js` precisam existir em `site/index.html`: `#graph`, `#family-nav`, `#characters-grid`, `#stories`, `#person-dialog`, `#about-dialog`, `#search`, `#fit`, `#zoom-in` e outros. Remover um ID do HTML sem remover ou adaptar o JavaScript causa erro no carregamento.

Da mesma forma, alterar um atributo `data-*` exige atualizar o listener correspondente no final de `main.js`.

5.10 Esquecer os testes e o build

Antes de publicar, execute:

```
npm test
npm run build
git diff --check
```

Os testes protegem a lógica, mas não substituem uma conferência visual. O build deve terminar com `Site estático criado em dist/` e a pasta `dist/` deve conter uma cópia navegável de `site/`.

5.11 Alterar apenas `dist/`

`dist/` é saída gerada e está no `.gitignore`. Não edite essa pasta como fonte principal. Edite `site/` e rode `npm run build` novamente. Caso contrário, a próxima geração do build apagará sua alteração.

5.12 Manter documentação desatualizada

Se os dados, o logo, o link do podcast ou o fluxo de publicação mudarem, revise `README.md`, `SOURCES.md`, `TODO.md` e este guia. Documentação desatualizada é uma fonte comum de publicação incorreta.

6. Como rodar localmente

É necessário servir o projeto por HTTP, porque os módulos JavaScript usam `import`. Abrir `site/index.html` diretamente com `file://` pode ser bloqueado pelo navegador.

Com Python:

```
cd antatrama
python3 -m http.server 3000 --bind 0.0.0.0 --directory site
```

Ou, usando o script do projeto:

```
npm run dev
```

Depois abra `http://localhost:3000`.

Para validar os arquivos:

```
node --check site/js/main.js
node --check site/js/graph.js
npm test
npm run build
```

7. Como hospedar no GitHub Pages

Opção A — publicar diretamente a raiz do repositório

Essa é a opção mais simples para este projeto.

1. Crie um repositório no GitHub. Se o site for público, lembre que imagens e dados enviados ficarão públicos.
2. Extraia o projeto mantendo esta estrutura: `index.html` na raiz e a pasta `site/` ao lado dele.
3. Envie os arquivos para a branch `main`.
4. No repositório, abra **Settings** → **Pages**.
5. Em **Build and deployment**, selecione **Deploy from a branch**.
6. Selecione a branch `main` e a pasta `/(root)`.
7. Clique em **Save** e aguarde o GitHub gerar o endereço.

O endereço normalmente será:

```
https://SEU-USUARIO.github.io/NOME-DO-REPOSITORIO/
```

A raiz contém um `index.html` que encaminha para `site/`. Os caminhos internos usam `./`, por isso funcionam quando o repositório é publicado em uma subpasta.

Opção B — publicar a pasta `dist`

Se preferir publicar a saída gerada:

```
npm run build
```

Depois, configure o GitHub Pages para usar a branch e a pasta que contenham a saída estática. A opção mais previsível, sem GitHub Actions, é publicar a raiz do repositório com o `index.html` de entrada. Se escolher uma pasta diferente, confirme que ela contém seu próprio `index.html` e todos os diretórios `css`, `js` e `assets`.

Envio por Git

Depois de criar o repositório vazio:

```
git init
git add .
git commit -m "Publicar aplicação genealógica Pantanal"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/NOME-DO-REPOSITORIO.
git push -u origin main
```

Se o projeto já tiver um repositório remoto configurado, não execute `git remote add origin` novamente. Confira primeiro:

```
git remote -v
git status
```

Nunca force-push (`git push --force`) para corrigir uma divergência sem entender o que existe no remoto. Preserve alterações de outras pessoas e resolva conflitos explicitamente.

Atualizações futuras

Após editar os arquivos:

```
npm test
npm run build
git add site README.md SOURCES.md TODO.md GUIA-TECNICO.md
git commit -m "Atualizar conteúdo da aplicação"
git push origin main
```

O GitHub Pages normalmente publica a nova versão automaticamente após o push. Pode ser necessário aguardar alguns minutos e atualizar o navegador sem cache.

Domínio de projeto e caminhos relativos

Para um repositório chamado `pantanal-antas`, o site geralmente ficará em `/pantanal-antas/`. Por isso, preserve os caminhos relativos atuais, como `./css/style.css`, `./js/main.js` e `./assets/...`. Não troque esses caminhos por `/css/style.css`, pois o caminho absoluto pode procurar os arquivos no domínio raiz e quebrar em páginas de projeto.

8. Checklist antes de publicar

- `index.html` permanece na raiz.
- `site/index.html` existe.
- `site/css/style.css` existe e inclui as regras mobile.
- `site/js/data.js` não contém nomes desconhecidos nas uniões.
- `site/js/graph.js` mantém as ordens visuais desejadas.
- Todas as imagens usadas têm caminhos relativos válidos.

- `npm test` termina sem falhas.
- `npm run build` termina sem falhas.
- `git diff --check` não mostra espaços problemáticos.
- O site foi conferido em uma largura próxima de 390 px e em uma largura desktop.
- Fotos e dados têm autorização para publicação.
- O link do Podcast ainda é provisório e deve ser substituído quando o endereço real do Spotify existir.

9. Limites atuais conhecidos

A aplicação não possui banco de dados nem edição colaborativa. Alterações são feitas no código e publicadas por Git. O cálculo de parentesco representa vínculos registrados no arquivo, não uma reconstrução científica completa da população. Ausência de um vínculo no documento não prova que ele não exista biologicamente.

O botão Podcast usa atualmente um endereço provisório no domínio do Spotify. Quando o episódio ou programa estiver publicado, substitua apenas o valor de `href` do elemento `.podcast-button` em `site/index.html`, mantendo o `target`, o `rel` e o ícone.